

RetroC

Language Reference Manual

A lightweight scripting language that compiles to Three-Address Code

1. Introduction

RetroC is a lightweight, high-level programming language designed to emulate the simplicity and efficiency of early computing systems. It serves as a scripting language that compiles into Three-Address Code (TAC), which represents instructions for a simulated retro CPU environment.

The main objective of RetroC is to provide a minimal yet expressive programming model suitable for demonstrating compiler design concepts such as lexical analysis, parsing, semantic analysis, and intermediate code generation.

RetroC follows a structured programming paradigm and supports essential constructs such as variables, control flow statements, functions, and arrays.

2. Lexical Structure

The lexical structure defines how source code is divided into meaningful units called tokens.

2.1 Tokens

Tokens are the smallest elements of the language.

Keywords

Reserved words with predefined meanings:

```
int, bool, char, true, false, if, else, while, for, return
```

Identifiers

Names used for variables and functions.

- Must start with a letter
- Can contain letters and digits

Format:

```
[a-zA-Z][a-zA-Z0-9]*
```

Integer Literals

A sequence of digits:

```
[0-9]+
```

Character Literals

A single character enclosed in single quotes:

```
'a', 'b', 'x'
```

Operators

Arithmetic, relational, and logical operators:

Category	Operators
Arithmetic	+ - * /
Assignment / Equality	= == !=
Relational	< > <= >=
Logical	&& !

Punctuation Symbols

```
; , ( ) { } [ ]
```

2.2 Whitespace and Comments

- Whitespace (spaces, tabs, newlines) is ignored.
- Comments improve readability and are ignored by the compiler.

Single-line comment:

```
// This is a comment
```

Multi-line comment:

```
/* This is  
a multi-line comment */
```

3. Syntax Specification (Grammar)

RetroC uses Extended Backus-Naur Form (EBNF) to define its syntax.

3.1 Program Structure

A program consists of multiple declarations:

```
<program> ::= <declaration>*
```

3.2 Declarations

```
<declaration> ::= <var_decl> | <func_decl>
```

3.3 Variable Declaration

```
<var_decl> ::= <type> <identifier> ("[" <int_literal> "]")? ";"
```

3.4 Data Types

```
<type> ::= "int" | "bool" | "char"
```

3.5 Function Declaration

```
<func_decl> ::= <type> <identifier> "(" <param_list>? ")" <block>
```

3.6 Parameters

```
<param_list> ::= <param> ("," <param>)*  
<param>      ::= <type> <identifier>
```

3.7 Block Structure

```
<block> ::= "{" <statement>* "}"
```

4. Statements

Statements define executable instructions.

```
<statement> ::= <var_decl>
               | <assignment> ";,"
               | <func_call> ";,"
               | <if_stmt>
               | <while_stmt>
               | <for_stmt>
               | <return_stmt> ";,"
               | <block>
```

4.1 Assignment

```
<assignment> ::= <identifier> "[" <expression> "]"? "=" <expression>
```

4.2 Conditional Statement

```
<if_stmt> ::= "if" "(" <expression> ")" <statement> ("else" <statement>)?
```

4.3 While Loop

```
<while_stmt> ::= "while" "(" <expression> ")" <statement>
```

4.4 For Loop

```
<for_stmt> ::= "for" "(" <var_decl_or_assign> ";," <expression> ";," <assignment> ")" <statement>
```

4.5 Return Statement

```
<return_stmt> ::= "return" <expression>?
```

5. Expressions

Expressions follow operator precedence rules.

```
<expression> ::= <logical_or>
```

Operator Precedence Hierarchy (highest to lowest):

Level	Operation	Operators
1 (lowest)	Logical OR	
2	Logical AND	&&
3	Equality	==, !=
4	Relational	<, >, <=, >=
5	Arithmetic	+, -, *, /
6 (highest)	Unary	!, - (negation)

6. Function Calls

```
<func_call> ::= <identifier> "(" <arg_list>? ")"
```

7. Type System & Semantic Rules

RetroC uses a static and strongly typed type system.

7.1 Supported Types

Type	Description	Example Values
int	Integer values	0, 1, 42, -10
bool	Boolean true/false	true, false
char	Single character	'a', 'z', 'X'

7.2 Semantic Rules

- Variables must be declared before use.
- No implicit type conversion between int and bool.
- Array index must be of type int.
- Conditions in if/while/for must evaluate to bool.
- Function return type must match the declaration.

7.3 Scope Rules

- Block-level scoping is used.
- Inner blocks can shadow outer variables.
- Symbol table manages scope using push/pop operations.

8. Memory Model

RetroC uses a stack-based memory model.

8.1 Storage

Variable Type	Storage Location
Local variables	Stack
Global variables	Static memory
Temporary (TAC) variables	Local memory

8.2 Execution

- Each function call creates a new stack frame.
- Temporary variables generated during TAC are stored in local memory.
- Stack frames are destroyed upon function return.

9. Intermediate Representation (TAC)

RetroC compiles source code into Three-Address Code (TAC). Each TAC instruction has at most one operator and three operands.

9.1 TAC Example

Source code:

```
a = b + c;
```

Generated TAC:

```
t1 = b + c  
a = t1
```

9.2 TAC Instruction Types

Instruction	Description
t1 = b + c	Binary arithmetic / logical operation
t1 = -b	Unary operation
a = t1	Simple copy / assignment
a[i] = t1	Array element write
t1 = a[i]	Array element read
if t1 goto L	Conditional branch
goto L	Unconditional jump
call f, n	Function call (n arguments)
return t1	Return from function

10. Example Programs

10.1 Factorial Program

Computes the factorial of an integer n using a for loop.

```
int factorial(int n) {
    int result;
    result = 1;
    for(int i = 1; i <= n; i = i + 1) {
        result = result * i;
    }
    return result;
}
```

10.2 Array Example

Declares a character array and initializes each element individually.

```
int main() {
    char letters[3];
    letters[0] = 'a';
    letters[1] = 'b';
    letters[2] = 'c';
    return 0;
}
```

10.3 Conditional Example

Demonstrates if-else branching and boolean expressions.

```
int main() {  
    int x = 10;  
    bool isLarge = x > 5;  
    if (isLarge && true) {  
        x = x + 10;  
    } else {  
        x = x - 5;  
    }  
    return x;  
}
```

11. Error Handling

RetroC handles errors in three distinct phases of compilation.

Phase	Error Types	Recovery Strategy
Lexical	Invalid characters, malformed tokens	Reported with line number; scanning continues
Syntax	Unexpected tokens, malformed structure	Panic-mode recovery using semicolon (;)
Semantic	Type mismatches, undeclared variables, invalid function calls, array index errors	Error reported; analysis continues where possible

12. Conclusion

RetroC is a simplified programming language designed for educational purposes, particularly for understanding compiler construction. It demonstrates key concepts such as lexical analysis, syntax parsing, semantic checking, and intermediate code generation using Three-Address Code (TAC).

Its minimal design makes it easy to implement while still supporting essential programming constructs including variables, arrays, control flow, and functions. RetroC serves as an ideal vehicle for learning the full compilation pipeline from source text to executable intermediate representation.